

The Prevalence and Impact of Licenses in Open Software Projects

Anonymous Author(s)

Abstract

The terms of how publicly available source code can be used are dictated by its license. The license (or its absence), in turn, affects what code the project may reuse and how its code can be (re)used and may also affect external participation and overall activity of the project. We aim to better understand the general state of license distribution overall and within language ecosystems and to investigate if license changes are associated with a noticeable variations of project output. To accomplish that we identify licenses and license types for over 100M software projects and find that most do not contain any license, that permissive licenses represent the bulk of most licenses, and that permissive licensing is representing an increasing proportion of all licenses over time. Restrictive licenses are more likely to be retained, however. There is a great variation among language ecosystems with C-language strongly favoring restrictive licenses. The analysis of license change impact comparing activity within one year of the adoption of the initial and final licenses shows that the change from restrictive to permissive license varies with the ecosystem. C-language ecosystems show reduced activity while Python shows increased activity when comparing restrictive to permissive license transition. Our results demonstrate dramatic changes in license type prevalence over time and find that the effects of license changes may have opposite effects depending on the language ecosystem.

1 Introduction

Open Source Software (OSS) is central to modern software development, fostering innovation, knowledge exchange, and the widespread adoption of libraries and tools. The licensing of OSS projects dictates how software can be used, modified, and distributed. This, in turn, should affect various aspects of projects' performance, such as new joiners, coding activity, the choice of upstream projects to depend upon, or the use by downstream projects.

Our first aim is to better understand the actual state of license selection in public software projects, including the choice of no license. Such unlicensed repositories play a significant role in the broader ecosystem, e.g., they contribute code substantially to larger projects [20] and they represent a significant portion of the massive training data for LLMs [18, 46, 47], emphasizing their importance in understanding reuse patterns.

The absence of licenses introduces legal uncertainties that may hinder downstream reuse. The problem may be propagated further, as the active and popular projects are often upstream dependencies for numerous projects downstream. Hence if such inappropriate use is detected, it can affect a very large part of the OSS ecosystem, not unlike what happened with *leftpad* [5]. Prior work found relatively low frequency of repositories with no license (e.g. 21.12% in [44] and 10.51% in [7]), that, as we demonstrate, greatly underestimates the proportion of projects with no license due to heavily filtering based on size, maturity, popularity, and activity.

In OSS projects, decisions are often made based on perceived supply chain concerns, such as choosing to participate in a project

that will become popular, hoping to have a large number of direct or transitive downstream users (and followers), selecting upstream dependencies that appear healthy, responsive, and active [8, 32, 33], and so on. License choice should have clear and far reaching effects on the supply chains in terms of attracting and retaining participants, having downstream dependents or users, and, of course on ways the code may be (re-)used. Despite this, we are not aware of any prior studies that model the impact of license choice nor any comprehensive theories that would explain it.

Our second aim is, therefore, to establish and test a preliminary theory that helps explain the impact of license choice in open software. The theory could help explain some of the reasons why a large fraction of projects have no license, why some license violations (such as the use of unlicensed code) appear to be tolerated, and what to expect when a developer (or an organization) changes a license. Such a theory can potentially lead to better tools that help developers choose the most suitable license according to their objectives, increase the proportion of projects with a license (hence spurring innovation that is presently inhibiting actors who do pay attention to licensing terms from using unlicensed code), and, more generally, reduce risks arising from license incompatibilities in open software supply chains.

To accomplish our goals we start from reviewing literature in areas that may help us formulate a preliminary theory, such as OSS developer motivation, company involvement in OSS, existing theories on how technology and libraries are selected and spread, as well as literature on OSS licensing. We pay particular attention to phenomena that could be measured at scale in order to make our theory easily testable. We then utilize a curated list of OSS projects with identified licenses and license types for nearly the entire OSS ecosystem in order to capture not only the most popular projects but to include small and inactive projects that are less likely to have a license. Based on this large curated list of licenses and projects, we use our preliminary theory to operationalize license choice where theory predicts a certain effect on key project factors. We first test if certain types of licenses have longer retention rates based on predictions of our theory. We then employ a multivariate multiple regression model to test which of the theory-predicted contextual factors are impacted by license choice.

Our analysis of the types and prevalence of licenses across more than 131 million OSS projects, finds that 83% of these projects lack a formal license. Among the licensed projects, the MIT license was the most commonly used, accounting for 65% of all licensed projects, followed by the Apache 2.0 and ISC licenses. Although permissive licenses dominate overall, copyleft licenses like the GPL exhibited high retention rates, reflecting a strong commitment to software freedom within certain communities. In comparison to prior studies we find four-fold larger prevalence of unlicensed code.

2 Theory Development

To theorize about the effects of license choice on a project's performance, several strands of prior research may be relevant. First is the

line of work on motivations of individuals [41] and businesses [17] to participate in OSS. Second, technical, social, and other factors that either constraint certain choices in software development or make them more convenient [32]. Third, the specific plans and objectives of the project, and how each license choice may advance and hinder these goals [36].

2.1 Motivations

In an extensive literature review, von Krogh et al. [41] categorized OSS participant motivations into ideology, altruism, kinship, fun, reputation, reciprocity, learning, own-use, career, and pay. Some of these motivations can probably influence the type of licenses contributors prefer. For instance, ideology and reciprocity may favor strongly copyleft licenses (over all other considerations), which require code sharing, while altruism may lean toward more permissive licenses. Kinship should drive adoption of licenses used by developers' social network, while own-use motivation appears to be orthogonal with license choice and may allow for other factors, like convenience and compatibility, to dominate.

In case of commercial involvement, motivations may diverge from individual contributors, even if they generally align with profit maximization [17]. Studies of open and gated software communities highlight the difficulties firms face in balancing community-based value creation with private value appropriation [37]. However, community sponsorship and licensing strategies can address these tensions. For instance, restrictive licenses can ensure long-term control over contributions, whereas nonmarket¹ sponsors might alleviate concerns about the project's sustainability without the need for restrictive licenses [39]. This highlights how the nature of the sponsor—market or nonmarket—may influence licensing strategies and developer motivations.

Empirical studies further emphasize the connection between license choice and project structure. Fershtman and Gandal [13] applied economic theories of motivation and found that factors such as status, signaling, or intrinsic motivation for participation in OSS projects are linked to restrictive licenses. They observed that restrictive licenses attract more contributors per project, while permissive licenses result in higher productivity per contributor. Similarly, research into firm involvement in OSS shows that licensing choices may be driven by strategic goals. For instance, firms may adopt permissive licenses to facilitate bundling proprietary and open source code [28]. Dahlander and Magnusson [9] identified three strategies firms use to engage with OSS communities: (1) accessing community development to extend resources, (2) aligning their strategy with community efforts, and (3) assimilating the community to integrate and share outcomes. Different commercial models necessitate distinct licensing strategies. Wagstrom et al. [42] and Zhou et al. [48] identified two types of firm engagement: community-focused firms (e.g., GNOME), which prioritize building vibrant OSS communities and monetize through services (which favor restrictive licenses so that a competitor can not create proprietary enhancements without sharing the code), and product-focused firms (e.g., Eclipse), which rely on product revenues and thus favors weaker copyleft provisions to allow some forms of bundling with proprietary code.

¹The term “nonmarket” excludes for-profit organizations.

Emergence of cloud services endangered the business model of OSS companies that rely on service contracts. Because cloud companies provide computing services, they can also run (and sell) the same OSS software as a service, thus cutting out companies that developed the software. For example, in 2021, Elasticsearch changed its Apache license to a commercial one where the products built from that code can not be provided to others as a managed service. They made another change in 2024, moving to Affero GPL, which is a copyleft license that requires anyone running a modified program on a server and letting other users communicate with it to also provide access to the source code of the modified version running on the server. As we can see, the more permissive license (Apache) was changed to a more restrictive one (commercial, then AGPL), as is typical for the service model.

Despite significant amount of research on the motivations to participate in OSS and commercial involvement, few of the key predictors can be reliably operationalized at scale. Based on the Fershtman and Gandal [13] conjecture that restrictive licenses attract more contributors per project, we can expect that adopting a restrictive license will result in higher contributor counts:

- **Hypothesis (H1a):** More restrictive licenses results in higher total numbers of authors.
- **Hypothesis (H1b):** More restrictive licenses (often ideologically motivated) are more likely to be retained.

2.2 Social and Technical Choice

Licenses, like other technologies and practices, spread through communities. Prior studies, such as those by Ma et al. [32] and others [26, 33], used the theory of social contagion [38] to estimate the impact of exposure, susceptibility, and infectiousness on developer choices regarding package (and tool) selection. These studies found that exposure measured by the number of overall deployments at the time of choice, had a significant positive effect on selection.

We expect a similar pattern for license selection. One dimension of exposure is the prevalence of a license (which is relatively constant over time) and favors the selection of popular licenses. i.e., developers who are indifferent or unaware of the differences between licenses are likely to gravitate toward those they have encountered. Just as developers tend to choose packages that are technically compatible with their existing technology stack and socially aligned with their collaborators' choices, we anticipate a similar behavior in license choice. Developers are likely to select a license compatible with their current projects or one already used by their collaborators (also, see “Kinship” motivation above).

The choice of programming language also influences license selection due to the culture and practices embedded within specific developer communities. For example, languages like Python and JavaScript, commonly used in web development, often favor permissive licenses like MIT or Apache, which emphasize reuse and flexibility [29]. In contrast, languages such as C or C++, which are dominant in systems programming, tend to be linked with copyleft licenses like GPL, prioritizing keeping derivative works open [14]. The language captures the influence of community norms and technical compatibility on license selection.

The community size associated with a project can also impact license decisions. Tsay et al. [40] found that projects with a large number of forks are perceived as more valuable and trustworthy

by the community. Projects aiming to encourage collaboration and reuse to maintain this momentum may favor permissive licenses. Similarly, Borges and Valente [3] suggest that the number of stars signals community approval, which, like forks, may influence the choice of more permissive licenses.

Furthermore, the number of upstream/downstream projects a project interacts with can play a crucial role in license selection. If a project interacts with many downstream projects, it may opt for licenses that balance flexibility and control, favoring weaker or conditional licenses over highly permissive or restrictive ones.

The decision to count upstream/downstream projects is motivated by the need to quantify a project's external interactions. While analyzing the specific licenses of these projects would provide more detailed insights, counting them offers a practical approach to estimate the complexity and potential challenges related to license alignment. Based on this discussion, we hypothesize that:

- **Hypothesis (H2a):** The license choice is affected by overall popularity of the license at adoption time.
- **Hypothesis (H2b):** The license choice is affected by programming language culture-specific norms.
- **Hypothesis (H2c):** More permissive licenses result in higher community size (i.e., upstream and downstream projects).

2.3 Project Goals

While legal compliance sets essential boundaries, the specific needs and objectives of a project further influence the selection of an appropriate license. Developers often choose licenses based on their preferences for the future use of the software. Research by Sen et al. [36] highlights that developers' motivations – such as a desire for widespread adoption or strong copyleft enforcement – also play a significant role in license selection.

The choice of license can significantly influence a project's development trajectory. Colazo and Fang [6] found that copyleft licenses promote greater permanence and long-term involvement in projects, affecting both developer contributions and engagement.

The number of commits is a key indicator of project activity and maintenance. Koch and Schneider [25] suggests that projects with frequent commits tend to have more active development cycles, making them more reliable and attractive for reuse. As project activity increases, maintainers may select licenses that either encourage contributions or protect the evolving codebase from potential misuse, balancing the needs for stability and control.

The number of files in a project reflects its complexity and modularity. Mockus [34] observed that larger projects with more files are more likely to offer reusable components, making license choice critical for balancing reuse and intellectual property protection. Bird et al. [2] further suggested that projects with more files tend to have modular structures, which encourage community engagement and adoption, potentially favoring more restrictive licenses.

Time-based metrics – such as the earliest and latest commits and the number of active months – provide insights into a project's maturity and stability. Capiluppi et al. [4] noted that older, more mature projects are often seen as more reliable, which could be influenced by the choice of licenses that support long-term sustainability, such as copyleft licenses. On the other hand, newer projects may opt for permissive licenses to facilitate rapid contributions during active development phases.

Burstiness, as a proxy for fluctuating activity, was operationalized by measuring the number of months in which the project had activity over its lifetime (from earliest to latest commit). Projects with high burstiness may not focus heavily on legal concerns, and instead tend to choose well-known, widely used licenses:

- **Hypothesis (H3a):** The license choice will affect the project activity approximated by number of commits.
- **Hypothesis (H3b):** More complex projects (higher number of files and blobs) will favor restrictive licenses.
- **Hypothesis (H3c):** Restrictive licenses support long-term sustainability as approximated by project activity duration.
- **Hypothesis (H3d):** Burstiness reflects the lack of stability and favors more permissive licenses.

3 Comparisons to Prior Work

Despite legal importance of license choice and the potential to stunt innovation and poison supply chains, it appears that a significant portion of OSS participants are ignoring the need to choose a license or to verify license compatibility. We construct a preliminary theory of how license choice might affect the project by reviewing relevant literature, operationalizing potential affected factors, and testing that theory. However, while our work is not the first to empirically study software licenses, it differs from the literature in two important ways.

Comprehensive Identification of Licenses. Most prior research, such as that by Wu et al. [44] and Xu et al. [45], primarily depends on explicit license declarations found in metadata files. Others, like Feng et al. [12], apply static analysis on binaries to detect embedded license texts. However, these methods may overlook licenses that are not clearly stated or are stored in unconventional directories. In contrast, we use a comprehensive dataset by Jahanshahi et al. [19], compiled by scanning virtually the entire OSS ecosystem for any files containing “license” in their filepath. Moreover, the authors use the winnowing algorithm, a robust method for matching license texts to known licenses, enhancing the accuracy of detecting both partial and full matches, even when the text is embedded or slightly modified. This method captures not only standard license files but also other files potentially holding licensing details, ensuring that no relevant license information is missed.

Scale and Scope of Analysis. Previous works often limit their scope to specific platforms (e.g., GitHub), a few package manager environments (e.g., NPM), or types of licenses (e.g., OSI-approved licenses), lacking a comprehensive, large-scale approach to detecting and analyzing licenses. Our study expands this scope by analyzing essentially the entire open source landscape, ensuring a more comprehensive cross-platform understanding of licensing practices.

In Table 1, we compare our study's methodology, coverage, scope, and scale with the most recent comparable studies in the field.

4 Methodology

To test our hypotheses, we adopt a two-step methodology. First, we gather descriptive statistics to quantify the prevalence, retention, and temporal trends of different license types within publicly available repositories. We then identify projects that switched from permissive to restrictive licenses (or vice versa) and measure their performance changes before and after the switch, using a multivariate multiple regression approach.

Table 1: Latest Studies in Open Source Software Licensing

	License Detection	Coverage	Scope	Scale (Projects)
[44]	Package Manager API	License Files	Maven, NPM, PyPI, RubyGems, Cargo	3,474,778
[7]	NLP/Extract License Terms	All Files	GitHub - >1000 Stars	16,341
[43]	GitHub API+NLP/Extract License Terms	All Files	GitHub - Cataloged by OpenHub	1,000
[45]	NLP/Extract License Terms	All Files	GitHub - Popular Python Projects	1,846
.....				
This Study	NLP/Matching Known Licenses	License Files	Entire OSS	131,171,379

4.1 License Types

We group licenses based on their characteristics. This grouping helps categorize and understand the different ways software can be distributed and modified. These categories typically include permissive, copyleft, weak copyleft, and public domain/unlicensed code. This classification is widely recognized in the field [21] and is supported by various scholarly sources.

1. *Permissive Licenses*: These licenses, such as the MIT and BSD licenses, are known for their minimal restrictions on how the software can be used. They allow software to be freely used, modified, and redistributed, even as part of proprietary software [22]. This permissiveness promotes wider adoption and integration of the software in diverse projects, including commercial applications [23].

2. *Copyleft Licenses*: These licenses, like the GNU General Public License (GPL), require that any modified versions of the software must also be distributed under the same license. This ensures that derivative works remain free and open, thus preserving the original freedoms granted by the license [11]. This characteristic is essential for maintaining the open source nature of software, as it prevents proprietary modifications [27].

3. *Weak Copyleft Licenses*: These licenses, such as the GNU Lesser General Public License (LGPL) and Mozilla Public License (MPL), strike a balance between permissive and strong copyleft licenses. They allow linking with proprietary software under certain conditions, while modifications to the licensed components themselves must remain open [1]. This flexibility encourages the use of open source libraries in both open and closed source projects [16].

4. *Conditional Open Licenses*: These licenses include specific conditions that must be met for usage, such as attribution (CC-BY), non-commercial use, or share-alike requirements. Creative Commons licenses, like CC-BY and CC-BY-SA, provide a framework for sharing while respecting the creator's conditions [10].

5. *Public Domain/Unlicensed*: Public domain and unlicensed software, including those using the Creative Commons Zero (CC0) license, are not restricted by copyright law. The authors of such software waive all rights, allowing anyone to use, modify, and distribute the work without any conditions. This category provides the maximum freedom for software use and is often utilized for simple, non-critical software components.

4.2 World of Code Infrastructure

World of Code (WoC)² is an infrastructure designed to cross-reference source code change data across the entire FLOSS community, facilitating sampling, measurement, and analysis within and across

²<https://worldofcode.org>

software ecosystems [30, 31]. In essence, it is a software analysis pipeline that encompasses the discovery and retrieval of data, storage and updates, and the transformations and data augmentation required for downstream analytic tasks [31].

WoC offers various maps that connect git objects and metadata (commits, blobs, authors) to each other. It also provides higher-level maps, such as project-to-data connections (e.g., project-to-author), author aliasing [15], and project deforking maps [35]. We use the project-to-license (P2L) map [19] from WoC that provides all the times at which a license was committed to a project and also verifies whether it still exists in the project's latest version.³ We also employ the concept of deforked projects as introduced by [35] to avoid potential biases from forks and duplicates of the same project. Throughout this paper, the term "project" refers to this deforked project unless stated otherwise.

4.3 License Change

We use license changes within public repositories to test our hypotheses on how license choice affects project performance. A project's decision to switch licenses should constitute a highly deliberate, informed event, typically involving careful consideration of implications related to legal constraints, developer participation, and user adoption. Unlike initial license selections – which may be arbitrary or uninformed – license changes reflect strategic intent, often in response to evolving community needs, competitive pressures, or growth ambitions. This intentionality makes license changes ideal natural experiments, enabling clearer, quasi-causal inferences regarding the impact of licensing decisions on measurable performance metrics. By focusing specifically on projects that have altered their licenses, this study leverages within-project variation, inherently controlling for stable project characteristics, thereby significantly strengthening internal validity and enhancing the precision of conclusions drawn from the analysis.

For example, moving to a more restrictive license type might be chosen to protect the project's intellectual property or to enforce open-source principles more rigorously. Kechagia et al. [24] illustrate how different license types affect dependency management and compliance in software projects.

To investigate the impact of license switching, we first identify projects with more than one license throughout their lifetime. We then refine this group to those that adopted exactly one license at their initial license adoption time and maintained exactly one license at their latest recorded status, thereby excluding any projects that held multiple licenses simultaneously at either point in time.

³Version V, latest at the time of this study.

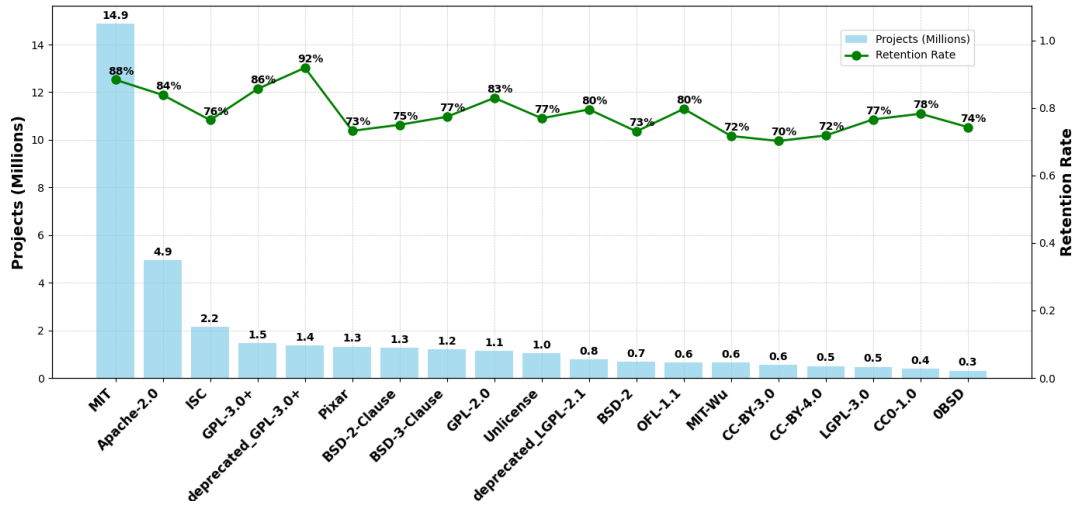


Figure 1: The distribution and retention rates of the most used licenses.

This also means that if a project starts with license A, then changes to B, and later changes to license C, we consider it as one change from A to C, rather than two separate changes. Consequently, we excluded projects where the first license type and the latest license type were the same. Although these projects had other licenses at some point, they were removed, resulting in no change between the first and last license types.

Next, we categorize licenses into two high-level groups: “permissive”, which includes public-domain and permissive licenses, and “restrictive”, encompassing copyleft, weak-copyleft, and conditional open licenses. Within these groups, we select projects that switched from a permissive to a restrictive license or vice versa.

To measure performance changes, we compute key project metrics during two separate one-year intervals: one immediately following the first license adoption and another immediately following the last license adoption. Projects are excluded if less than one year separates the adoption dates of the first and last licenses or if less than one year has elapsed since the last license adoption at the time of our data curation. We then model the change in these metrics (i.e., metric values in the post-switch interval minus those in the pre-switch interval) based on the direction of the license change (restrictive to permissive or permissive to restrictive). We control for the project’s primary language, its start and end times, the delay before the first license adoption, the elapsed time between the first and last license adoptions, and the proportion of new OSS projects adopting that license type at the time of the last license change.

To test our hypotheses, we selected a regression model appropriate to the nature of our response variable, specifically, multivariate multiple regression. This model is an extension of multiple regression that allows for multiple dependent variables (outcomes) to be predicted by multiple independent variables (predictors) simultaneously. This model is useful when the dependent variables are correlated and need to be analyzed together to improve prediction accuracy and interpretation.

5 Results and Discussion

We organize our results in three parts. First, we present a broad exploration of different license types, highlighting their prevalence, retention, and general trends. Next, we detail the regression analysis that examines how the choice of license relates to project performance. Finally, we focus on the phenomenon of license switching by analyzing performance metrics before and after a change from permissive to restrictive (or vice versa).

5.1 Types and Prevalence of Licenses

5.1.1 Our Findings. We first count how many projects had at least one occurrence of any license during their lifetime, then identify the top 20 most used licenses, as shown in Figure 1, and analyze their frequency. Out of the 131,171,379 projects indexed by WoC, 22,281,342 projects had at least one license committed to their project at some point in their lifetime. However, when examining the latest version of these projects, this number drops to 20,110,256 projects. When interpreting the numbers in the figure, note that the sum of per-license counts exceeds the total number of projects with a license. This discrepancy arises because some projects have multiple licenses and are therefore counted in several categories.

To calculate the license retention rates, we count how many projects still had the license in their latest state and divide this number by the total number of projects that had ever used the license. This retention rate, also in Figure 1, indicates the proportion of projects that continue to use the license over time.

As shown in the figure, the MIT license is the most prominent, followed by the Apache 2.0 and ISC licenses. Notably, per H1b, the deprecated GPL 3 license (with strong copyleft provisions) has the highest retention rate. In contrast, the Creative Commons 3 license (CC-BY-3.0) has the lowest retention rate.

We also analyze the distribution of projects according to their license types, in two conditions. First, we consider the distribution across the entire project history, meaning that if a project adopted a license at any point and later removed it, it is still counted as having

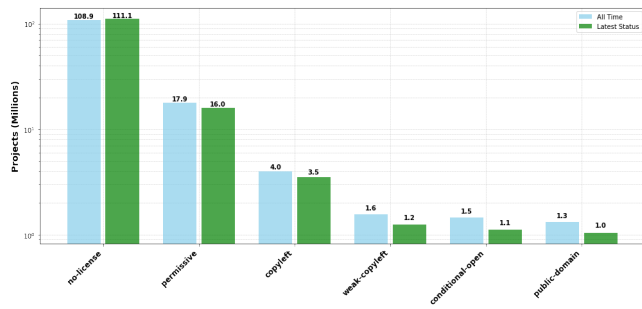


Figure 2: The distribution of license types across projects.

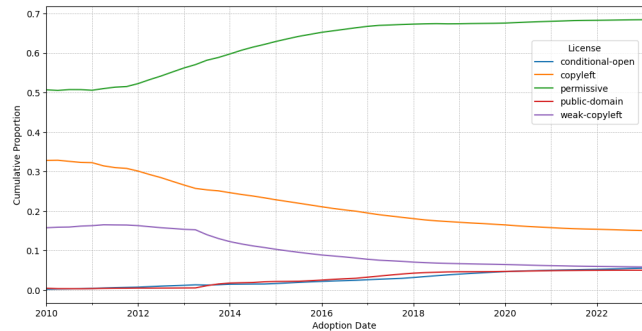


Figure 3: Proportion of license type over time

that license. Second, we determine the distribution based on the most recent status of each project. The results in Figure 2 show that “conditional-open”, “weak-copyleft”, and “public-domain” license types have the lowest retention, with approximately a quarter of the projects that ever had such license changing it by their latest version. For comparison, only approximately 8% of “copyleft” licenses were changed, which is consistent with our theory (H1b) that ideology-related license choice should be most “sticky”.

We also analyze the proportion of adopted licenses at each point in time, categorized by license type, to identify trends in licensing preferences over time. This allows us to observe shifts in the adoption of different licenses and assess whether certain license types have gained or declined in popularity. By tracking these proportions longitudinally, we can better understand how licensing choices evolve and whether external factors, such as regulatory changes or community norms, influence these trends. The results are shown in Figure 3. Looking at the trends we clearly see that although copyleft licenses may be the most sticky, the tendency to adopt these licenses has decreased over time (H2a).

5.1.2 Comparison with Literature. Table 2 provides a comparison between the results in the literature and our study, highlighting substantial differences primarily due to variations in scope, methodology, and dataset size (see Table 1).

A key methodological difference between our study and Wu et al. [44] lies in the treatment of data points. Wu et al. [44] considered each version of a package as a separate data point, resulting in 46.59 million data points from 3.47 million projects. This approach could overrepresent projects with many versions, potentially skewing

adoption trends. Our study treats each project as a single data point, providing a more balanced view of the ecosystem without overemphasizing projects with frequent updates.

The focus of the Wu et al. [44] study on only five package managers might explain the significant discrepancy in the percentage of projects without a license: 21.12% in their case,⁴ versus 83.01% in ours. On the other hand, Cui et al. [7] reported that only 10.51% of projects were unlicensed. The difference with Wu et al. [44] likely stems from the fact that the package managers they examined are associated with more mature and well-maintained projects, which are more likely to have defined licenses. Additionally, Cui et al. [7] filtered their data to include only projects with over 1,000 stars, which might explain why the percentage of unlicensed projects is so low.⁵ In contrast, our dataset is broader, including projects which may be less mature or less formally governed, leading to a higher incidence of unlicensed projects.

The analysis of copyleft versus permissive licenses further reflects the differences in scope. Wu et al. [44] found a preponderance of permissive licenses, with much lower percentages for copyleft and weak copyleft licenses. However, our study, covering all programming languages, reveals a more diverse licensing landscape, with a lower percentage of permissive licenses and higher percentages of copyleft licenses. This suggests that by focusing on just five package managers, Wu et al. [44] might underrepresent broader licensing trends, particularly the prevalence of copyleft licenses across the global open source ecosystem.

Key Findings

- (1) Overall, 83% of the projects in the dataset lack a formal license, which is significantly higher than previously reported.
- (2) The MIT license is the most widely used license, accounting for 65% of all licensed projects and 10% of all public projects.
- (3) Despite the overall preference for permissive licenses, copyleft licenses such as GPL have high retention rates cf. H1b.
- (4) The differences in license usage reported in our study compared to prior work highlight how sensitive such analyses are to sampling strategies.

5.1.3 Implications. The analysis of public repository licensing reveals that 83% of projects lack a formal license, posing significant legal risks and discouraging innovation by preventing responsible actors from reusing unlicensed code. This highlights the need for developers to prioritize licensing to set clear terms of use. Our findings challenge previous findings, which underestimate the prevalence of unlicensed projects by focusing on mature ones, missing the broader risks of software reuse.

Even though many of these unlicensed projects may seem trivial and technically not OSS, their public availability still creates legal ambiguity and poses a risk of license noncompliance, especially when combined with the lack of thorough legal review in many downstream projects. Jahanshahi et al. [20] demonstrated that in

⁴Since the paper provides these percentages by package manager, we calculated a weighted average based on the number of projects per package manager.

⁵These could also be influenced by the differences between OSS projects and publicly available projects as was explained in the introduction.

Table 2: Comparison between our results and the literature.

	Projects	No License	Copyleft	Weak Copyleft	Conditional Open	Permissive	Public Domain
[44]	3,474,778	21.12%	5.72%	5.12%	-	89.16%	-
[7]	16,341	10.51%	11.32%	0.97%	-	87.71%	-
This Study	131,171,379	83.01%	15.35%	5.26%	4.82%	70.18%	4.39%

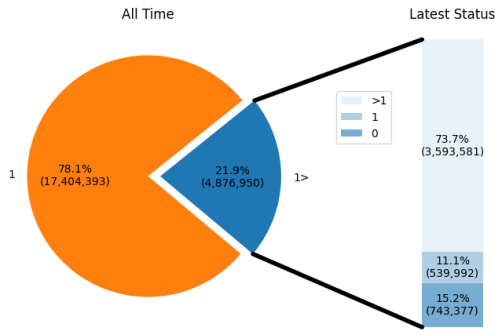


Figure 4: Distribution of license type count in projects

the context of copy-based reuse, nearly 18% of reused artifacts originated from very small projects,⁶ while large projects⁷ accounted for only 32% of reused artifacts. This highlights the importance of addressing the issue of unlicensed code within the OSS ecosystem.

Among projects that are licensed, the MIT license is the most prevalent, reflecting a historic preference for permissive licenses that facilitate broad adoption and collaboration. However, the continued use of copyleft licenses like the GPL (which was popular decades ago) demonstrates that many developers are committed to preserving the open nature of derivative works.

5.2 The Impact of License Changes

Examining the distribution of license type counts in projects (Figure 4), we observe that nearly 22% of projects had more than one license type in their lifetime. Among these, 74% still had more than one license type in their latest status as well, indicating they did not necessarily change their license. On the other hand, 15% of these projects do not have any license in their latest status, meaning they deleted their license. The middle group, consisting of projects that had more than one license type in their lifetime but have only one type ultimately, is the group we analyze license changes in.

5.2.1 Our Findings. Now we turn to the results of our regression model discussed in the Methodology section. The basic statistics of the response and control variables, including the 5th percentile, median, mean, and 95th percentile for the numeric variables, as well as the counts of different levels for the factor variables, are presented in Table 3.

Given that we have multiple response variables, the model provides a distinct set of coefficients for each outcome. Each set of

⁶Projects with no stars and fewer than 10 commits.

⁷Projects with more than 10 stars and 100 commits.

coefficients indicates the influence of the predictor variables on the probability of the response variable assuming that there might be correlation between response variables.

For language variable, we employed sum contrasts, also known as effect coding, where each level of the predictor variable is compared to the overall mean of all levels. This approach is particularly advantageous in models where the goal is to compare each category to the overall mean rather than to a specific reference category, as it offers a more symmetric interpretation of the effects. In the sum contrast method, the sum of the coefficients for all levels, including the intercept, must equal zero. We also include the interaction term between language and license change in the model.

The change direction variable has two levels, with Permissive to Restrictive (P2R) set as the base level. As a result, the model outputs only display coefficients for Restrictive to Permissive (R2P), which indicate the difference relative to the base level (P2R).

Furthermore, when interpreting the earliest and latest commit times, it is important to note that the predictors in the model represent the time elapsed since those commits. A higher earliest commit value indicates that the commit occurred further in the past, meaning the project is older, whereas a lower value suggests a more recent commit. Similarly, a higher latest commit value means the project has been inactive for a longer period.

Table 4 shows Type III MANOVA Tests with Pillai test statistic for the fitted model. The results show that the p-values for all the predictors are close to zero, indicating that each predictor significantly improves our model's fit.

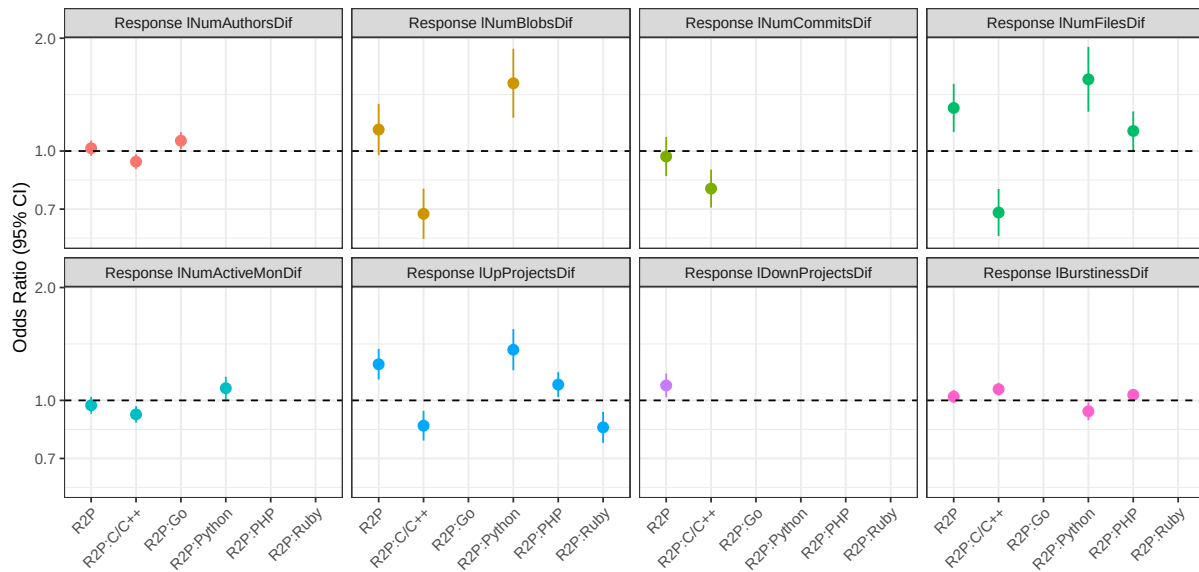
To better understand the effect of license change direction on project metrics, we calculate the odds ratios for the change direction predictor and its interaction with language, considering only those predictors that are statistically significant at the 95% confidence level. This ensures that the results focus on meaningful relationships rather than noise. Figure 5 displays these significant odds ratios along with their 95% confidence intervals.⁸ Since the analysis is based on a multivariate regression model, the odds ratios are presented across different output variables, allowing for a comparison of how the change direction predictor and its interaction with language influence various aspects of the model.

In these models, each coefficient represents a change in the log-odds (or log-count) of the outcome. The main R2P effect β_{R2P} shows how the outcome shifts when moving from a restrictive to a permissive license under sum contrasts (i.e., averaged across language levels). A significant R2P:Language interaction $\beta_{R2P:Lang}$ adds to (or subtracts from) the overall R2P coefficient for that specific language. To obtain the total restrictive→permissive effect for a given language, we sum $\beta_{R2P} + \beta_{R2P:Lang}$ and then exponentiate. The result

⁸Please refer to the replication package for the full set of coefficients and p-values.

Table 3: License change model - descriptive statistics.

	Variable	Description	Statistics			
			5%	Median	Mean	95%
Response	AuthorsDiff	Difference in number of authors	-5	0	0.91	6
	BlobsDiff	Difference in number of blobs	-1,470	-17	56.6	1,059.2
	CommitsDiff	Difference in number of commits	-309.1	-6	0.83	209
	FilesDiff	Difference in number of files	-856.1	-4	116.92	634
	ActiveMonDiff	Difference in number of active months	-8	-1	-1.06	5
	UpProjectsDiff	Difference in number of upstream projects	-35	-1	-3.54	16
	DownProjectsDiff	Difference in number of downstream projects	-7	0	19.68	25.1
	BurstinessDiff	Difference in burstiness (duration / active months)	-7	0.28	1.27	10.5
Control	EarliestCommit	Project age (months)	39.87	84	90.05	162.04
	LatestCommit	Project latest inactivity (months)	3.23	27.57	34.95	87.64
	AdoptDelay	Earliest commit to license adoption (months)	0	0.13	3.35	19.37
	Distance	Distance between first and last adoption (months)	12	24	30.77	73
	Proportion	Proportion of projects adopting same license at change time	0.05	0.64	0.44	0.68
	Language	JavaScript Python C/C++ Java PHP Go Ruby Other				
	Counts (%)	7,566 (32.03%) 3,708 (15.70%) 3,027 (12.82%) 2,371 (10.04%) 1,782 (7.54%) 526 (2.23%) 518 (2.19%) 4,121 (17.45%)				
	Change Direction	Restrictive to Permissive		Permissive to Restrictive		
	Counts (%)	14,329 (60.67%)		9,290 (39.33%)		

**Figure 5: License change model - odds ratios**

is the odds ratio – a multiplier on the odds of an increase in the outcome metric compared to the permissive→restrictive direction.

For the difference in the number of authors, C/C++ exhibits an odds ratio of about 0.95, suggesting these projects are roughly 5% less likely to see increased authorship after moving from a restrictive to a permissive license compared to the opposite shift. In contrast, Go shows an odds ratio of about 1.08, indicating an 8% greater likelihood of authorship growth under permissive licensing. Although these effects are relatively modest, they demonstrate clear language-specific differences in how communities respond to

changes in license direction. While this partially confirms H1a for C/C++ projects, it is not the same for all languages.

Regarding the difference in the number of blobs, C/C++ projects exhibit an odds ratio of about 0.78, indicating these projects are roughly 22% less likely to experience blob growth when shifting from restrictive to permissive licenses (compared to going permissive to restrictive). By contrast, Python shows an odds ratio of about 1.73, suggesting a 73% higher likelihood of increased repository blob counts under permissive licensing. The notable magnitude of these odds ratios emphasizes that licensing direction can strongly shape development dynamics in these languages.

Table 4: License change model - Type III MANOVA Tests: Pillai test statistic

Variable	DF	Test Stat	p.value
(Intercept)	1	0.024183	$< 2 \times 10^{-16}$
Change Direction	1	0.002403	1.99×10^{-9}
Language	7	0.012655	$< 2 \times 10^{-16}$
EarliestCommit	1	0.061345	$< 2 \times 10^{-16}$
LatestCommit	1	0.111041	$< 2 \times 10^{-16}$
Delay	1	0.028073	$< 2 \times 10^{-16}$
Distance	1	0.018359	$< 2 \times 10^{-16}$
Proportion	1	0.001388	6.79×10^{-5}
Change:Language	7	0.008897	$< 2 \times 10^{-16}$

For the difference in the number of files, the overall restrictive to permissive license shift has an odds ratio of about 1.30, indicating roughly 30% higher odds of increased file counts. However, this positive effect is notably reduced in C/C++, where the odds ratio is about 0.89 (around 11% lower odds). In contrast, PHP and Python have odds ratios near 1.47 and 2.02, respectively, suggesting that PHP projects are about 47% more likely and Python projects are about 100% more likely to experience file-count growth under permissive licensing.

C/C++ (often complex projects) reduced growth (blobs/files) after shifting to permissive licenses that suggests restrictive licenses were originally preferred to manage complexity effectively, partially supporting H3b.

In the difference in the number of commits, only C/C++ shows a statistically significant result, with an odds ratio of about 0.77—indicating a roughly 23% lower likelihood of increased commit frequency when shifting to a permissive license compared to the opposite direction. This moderate odds ratio suggests a tangible decrease in development activity within C/C++ communities following a move to permissive licensing, partially supporting H3a.

Concerning the difference in active months, C/C++ projects show an odds ratio of about 0.89, indicating roughly 11% lower odds of sustained activity after moving from a restrictive to a permissive license, partially supporting H3c. By contrast, Python has an odds ratio of about 1.05, suggesting around 5% higher odds of maintaining longer periods of active development under permissive licenses. Although these effects are modest, they illustrate clear language-specific variations.

Regarding the difference in upstream projects, the overall restrictive to permissive shift shows an odds ratio of about 1.25, indicating roughly 25% higher odds of increased reuse from upstream. The interactions for C/C++ and Ruby, though negative relative to the overall effect, still place these languages slightly above 1 (about 1.07 and 1.06, respectively). In contrast, PHP stands at about 1.38, and Python reaches about 1.70, suggesting considerably greater odds of upstream integration under permissive licensing. These variations across languages emphasize the nuanced ways in which license directionality can shape project dependencies.

For the difference in downstream projects, the overall effect of going from a restrictive to a permissive license is an odds ratio of about 1.10, indicating roughly 10% higher odds of increased downstream usage or integration. This moderate boost suggests

that adopting a permissive license tends to enhance a project's popularity within broader software ecosystems.

Lastly, for the difference in burstiness, the language-specific interactions point to moderate yet meaningful distinctions. C/C++ projects have an odds ratio of about 1.10 (around 10% greater odds of more intense activity bursts) under permissive licensing, while PHP stands at about 1.06 (6% higher odds). By contrast, Python's total odds ratio is about 0.96, indicating roughly 4% lower odds of experiencing bursty activity. Although modest, these differences highlight notable variations in development dynamics associated with licensing direction. This shows partial support for H3d, highlighting once again the nuances between different programming languages.

These quantified magnitudes, along with their directions, clearly demonstrate that license transitions influence project metrics meaningfully, with notable variations across different metrics and programming languages.

Key Findings

- (1) C/C++ projects see reduced authorship, commits, files, blobs, and activity after switching to permissive licenses cf. H2b.
- (2) Python projects see increased repository growth and sustained activity under permissive licenses cf. H2b.
- (3) PHP projects benefit from permissive licenses through increased file additions and external project integration cf. H2b.
- (4) Go projects attract moderately more authors after permissive license adoption cf. H2b.
- (5) Permissive licenses generally increase both upstream and downstream project adoption. cf. H2c.
- (6) Ruby projects show reduced upstream integration when adopting permissive licenses.
- (7) License transitions have a moderate effect on burstiness: permissive licenses increase bursty activity for C/C++ and PHP, but reduce it for Python.

5.2.2 Implications. Programming language implies a broader choice of technology, libraries, and tools. Thus, users of that entire ecosystem are probably nudged toward the most common licenses used within it. For example, C-based projects often prefer restrictive licenses, while Go and Python projects lean toward permissive ones. Developers should be aware of how their language community influences licensing norms, and the community could offer language-specific guidelines to align new projects with licensing best practices.

In the C/C++ community, where switching to permissive licenses slightly discourages new contributors, project maintainers might need to pair such license transitions with additional strategies – such as enhanced outreach or clearly defined governance policies – to retain or attract authors. Conversely, in the Go community, the positive response to permissive licenses indicates that maintainers could effectively use permissive licensing as a practical tool to encourage contributor participation, enhancing community building without significant additional effort.

Repository growth implications, measured by the number of blobs, are particularly impactful. Python communities experience notable growth under permissive licenses, indicating maintainers

can strategically leverage permissive licensing to attract broader contributions, resulting in richer functionality and greater innovation. For C/C++, however, the practical advice would be cautious—adopting permissive licenses without supplemental incentives might unintentionally discourage contributors from investing in substantial new content. Thus, maintainers in these communities should carefully weigh the trade-offs or introduce additional motivators (e.g., clearer contributor recognition or rewards) alongside permissive licensing decisions.

Since permissive licensing tends to lower commit frequency in C/C++, those community leaders considering permissive licensing should prepare targeted interventions, such as clearly articulated development roadmaps, contributor guidelines, or engagement incentives to sustain development momentum after a licensing change.

Implications for file-count differences indicate concrete strategic guidance. Given that PHP and Python projects substantially benefit from permissive licenses regarding file creation, maintainers in these ecosystems can confidently adopt permissive licenses when aiming for feature expansion or modularization. Conversely, C/C++ maintainers should be aware that permissive licenses may require additional supporting strategies—such as enhanced documentation or modular project structures—to ensure continued growth.

Practical considerations around project activity duration (active months) suggest nuanced strategies: Python projects benefit slightly in terms of sustained activity from permissive licenses, indicating that maintainers aiming for long-term, stable contributions can adopt permissive licenses confidently. In contrast, C/C++ maintainers should recognize that permissive licensing may slightly shorten sustained activity periods, making supplemental engagement strategies crucial – such as periodic contribution drives or community recognition – to sustain longer-term involvement.

Insights on upstream project usage have clear practical ramifications. Python and PHP communities' increased usage of external OSS projects after adopting permissive licenses suggests maintainers might strategically use permissive licensing to encourage their projects' broader integration into complex OSS ecosystems. Conversely, C/C++ and Ruby projects that see decreased usage of external dependencies under permissive licenses could benefit from using licensing strategically to foster self-contained, tightly controlled project environments—useful in contexts requiring high stability or predictable dependency management.

The moderate improvement in downstream project adoption following permissive licensing highlights practical licensing guidance for maintainers seeking wider adoption. Project maintainers aiming for their software to become foundational or widely used dependencies can practically adopt permissive licenses as a direct strategy to facilitate their projects' broader use across diverse OSS communities and applications.

Lastly, implications regarding burstiness—concentrated periods of intense development—offer actionable guidance. Since permissive licensing boosts burst-like activity in C/C++ and PHP, maintainers aiming for rapid or intensive development cycles (e.g., to meet release deadlines or achieve specific project milestones) could strategically employ permissive licensing to trigger and sustain such heightened activity. Conversely, Python communities, experiencing steadier activity under permissive licenses, may practically

leverage permissive licensing for achieving predictable and consistent development flows rather than periodic intensive bursts.

Together, these practical insights enable OSS maintainers to make informed licensing choices strategically tailored to specific community dynamics, desired project outcomes, and broader ecosystem goals.

6 Limitations

The license map (P2L) [19] used for our study relies on detecting license files committed to a project's repository. While this approach does not assume that licenses are consistently and accurately recorded in dedicated license.md files, it does not check the content of all files. In practice, licenses might be specified within individual source files, potentially leading to underreporting or misclassification of a project's licensing status.

Additionally, while the P2L map captures license changes over time, it may not fully account for licenses that were temporarily removed or altered before being reinstated, which could introduce inaccuracies in assessing a project's long-term licensing practices. The reliance on the latest status of licenses might also obscure important historical context, such as changes in licensing throughout a project's lifecycle, which could be crucial for understanding the project's evolution and the factors influencing license choices.

Despite the efforts of manual verification employed in P2L map, these limitations underscore the need for caution when interpreting results based on it. Complementary methods, such as cross-referencing with other data sources, may be necessary to obtain a more complete and accurate picture of licensing practices in open source software projects.

7 Conclusions

The study offers a discussion of theoretical concepts combined with rigorous statistical data analysis of the factors influencing license choices in open source software projects. Not all predictions were borne out, suggesting that prior work used to support the hypothesized effect was either no longer relevant or obtained in a different context. Key finding that over 80% of the projects have no license making their code legally virtually unusable highlight both risks of inadvertent poisoning of the software supply chain and, perhaps more importantly, the obstacle for innovation through its wider use. Many of the theorized predictions were supported by the analysis, yet there was almost always meaningful differences between different programming languages, suggesting that a more mature theory of license choice could be developed. The observed relationships may help develop recommenders that can provide projects with the most suitable default license and suggest changes as project matures or its goals change.

8 Data Availability

The replication package, including used datasets and R code are available at: <https://zenodo.org/records/15031139>.

References

- [1] Emad Alamoudi, Rashid Mehmood, Wajdi Aljudaibi, Aiiad Albeshri, and Syed Hamid Hasan. 2020. *Open Source and Open Data Licenses in the Smart Infrastructure Era: Review and License Selection Frameworks*. Springer International Publishing, Cham, 537–559. https://doi.org/10.1007/978-3-030-13705-2_22
- [2] Christian Bird, Nachiappan Nagappan, Harald Gall, Brendan Murphy, and Premkumar Devanbu. 2009. Putting it all together: Using socio-technical networks to predict failures. In *2009 20th International Symposium on Software Reliability Engineering*. IEEE, 109–119.
- [3] Hudson Borges and Marco Tulio Valente. 2018. What's in a github star? understanding repository starring practices in a social coding platform. *Journal of Systems and Software* 146 (2018), 112–129.
- [4] Andrea Capiluppi, Patricia Lago, and Maurizio Morisio. 2003. Characteristics of open source projects. In *Seventh European Conference on Software Maintenance and Reengineering, 2003. Proceedings*. IEEE, 317–327.
- [5] Md Atique Reza Chowdhury, Rabe Abdalkareem, Emad Shihab, and Bram Adams. 2021. On the untriviality of trivial packages: An empirical study of npm javascript packages. *IEEE Transactions on Software Engineering* 48, 8 (2021), 2695–2708.
- [6] Jorge Colazo and Yulin Fang. 2009. Impact of license choice on open source software development activity. *Journal of the American Society for Information Science and Technology* 60, 5 (2009), 997–1011.
- [7] Xing Cui, Jingzheng Wu, Yanjun Wu, Xu Wang, Tianyue Luo, Sheng Qu, Xiang Ling, and Mutian Yang. 2023. An empirical study of license conflict in free and open source software. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 495–505.
- [8] Laura A. Dabbish, H. Colleen Stuart, Jason Tsay, and James D. Herbsleb. 2012. Social coding in GitHub: transparency and collaboration in an open software repository. In *CSCW*. 1277–1286.
- [9] Linus Dahlander and Mats Magnusson. 2008. How do firms make use of open source communities? *Long range planning* 41, 6 (2008), 629–649.
- [10] Melanie Dulong de Rosnay. 2009. Creative commons licenses legal pitfalls: Incompatibilities and solutions.
- [11] Massimo D'Antoni and Maria Alessandra Rossi. 2007. COPYLEFT LICENCING AND SOFTWARE DEVELOPMENT.
- [12] Muyue Feng, Weixuan Mao, Zimu Yuan, Yang Xiao, Gu Ban, Wei Wang, Shiyang Wang, Qian Tang, Jiahuan Xu, He Su, et al. 2019. Open-source license violations of binary software at large scale. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 564–568.
- [13] Chaim Fershtman and Neil Gandal. 2007. Open source software: Motivation and restrictive licensing. *International Economics and Economic Policy* 4 (2007), 209–225.
- [14] Brian Fitzgerald. 2006. The Transformation of Open Source Software. *MIS Quarterly* 30, 3 (2006), 587–598. <http://www.jstor.org/stable/25148740>
- [15] Tanner Fry, Tapajit Dey, Andrey Karnauch, and Audris Mockus. 2020. A dataset and an approach for identity resolution of 38 million author ids extracted from 2b git commits. In *Proceedings of the 17th international conference on mining software repositories*. 518–522.
- [16] Jonas Gamalielsson and Björn Lundell. 2017. On licensing and other conditions for contributing to widely used open source projects: an exploratory analysis. In *Proceedings of the 13th International Symposium on Open Collaboration*. 1–14.
- [17] Dietmar Harhoff, Joachim Henkel, and Eric von Hippel. 2003. Profiting from voluntary information spillovers: how users benefit by freely revealing their innovations. *Research Policy* 32, 10 (2003), 1753–1769. [https://doi.org/10.1016/S0048-7333\(03\)00061-1](https://doi.org/10.1016/S0048-7333(03)00061-1)
- [18] Mahmoud Jahanshahi and Audris Mockus. 2025. Cracks in The Stack: Hidden Vulnerabilities and Licensing Risks in LLM Pre-Training Datasets. *arXiv preprint arXiv:2501.02628* (2025).
- [19] Mahmoud Jahanshahi, David Reid, Adam McDaniel, and Audris Mockus. 2025. OSS License Identification at Scale: A Comprehensive Dataset Using World of Code. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE.
- [20] Mahmoud Jahanshahi, David Reid, and Audris Mockus. 2025. Beyond Dependencies: The Role of Copy-Based Reuse in Open Source Software Development. *ACM Trans. Softw. Eng. Methodol.* (Jan. 2025). <https://doi.org/10.1145/3715907> Just Accepted.
- [21] Halina Kaminski and Mark Perry. 2007. Open source software licensing patterns. In *Computer Science Publications*. 10.
- [22] Georgia M Kapitsaki and Georgia Charalambous. 2019. Modeling and recommending open source licenses with findOSSLicense. *IEEE Transactions on Software Engineering* 47, 5 (2019), 919–935.
- [23] Georgia M Kapitsaki, Nikolaos D Tselikas, Kyriakos-Ioannis D Kyriakou, and Maria Papoutsoglou. 2022. Help me with this: A categorization of open source software problems. *Information and Software Technology* 152 (2022), 107034.
- [24] Maria Kechagia, Diomidis Spinellis, and Stephanos Androutsellis-Theotokis. 2010. Open source licensing across package dependencies. In *2010 14th Panhellenic conference on informatics*. IEEE, 27–32.
- [25] Stefan Koch and Georg Schneider. 2002. Effort, co-operation and co-ordination in an open source software project: GNOME. *Information Systems Journal* 12, 1 (2002), 27–42.
- [26] Hemank Lamba, Asher Trockman, Daniel Armanios, Christian Kästner, Heather Miller, and Bogdan Vasilescu. 2020. Heard it through the Gitvine: an empirical study of tool diffusion across the npm ecosystem. In *International Conference on the Foundations of Software Engineering (FSE)*. 505–517.
- [27] Philippe Laurent. 2012. Free and Open Source Software Licensing: A reference for the reconstruction of "virtual commons"? In *Conference for the 30th Anniversary of the CRID*. 1–19.
- [28] Josh Lerner and Jean Tirole. 2002. Some Simple Economics of Open Source. *The Journal of Industrial Economics* 50, 2 (Jun 2002), 197–234. <http://www.jstor.org/stable/3569837>
- [29] Josh Lerner and Jean Tirole. 2005. The economics of technology sharing: Open source and beyond. *Journal of Economic Perspectives* 19, 2 (2005), 99–120.
- [30] Yuxing Ma, Chris Bogart, Sadika Amreen, Russell Zaretski, and Audris Mockus. 2019. World of code: an infrastructure for mining the universe of open source VCS data. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 143–154.
- [31] Yuxing Ma, Tapajit Dey, Chris Bogart, Sadika Amreen, Marat Valiev, Adam Tutko, David Kennard, Russell Zaretski, and Audris Mockus. 2021. World of code: enabling a research workflow for mining and analyzing the universe of open source VCS data. *Empirical Software Engineering* 26 (2021), 1–42.
- [32] Yuxing Ma, Audris Mockus, Russel Zaretski, Randy Bradley, and Bogdan Bichescu. 2022. A Methodology for Analyzing Uptake of Software Technologies Among Developers. *IEEE Transactions on Software Engineering* 48, 2 (2022), 485–501. <https://doi.org/10.1109/TSE.2020.2993758>
- [33] Addi Malviya-Thakur, Audris Mockus, Russell Zaretski, Bogdan Bichescu, and Randy Bradley. 2023. How R Developers explain their Package Choice: A Survey. In *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–12. <https://doi.org/10.1109/ESEM56168.2023.10304869>
- [34] Audris Mockus. 2007. Large-scale code reuse in open source software. In *First International Workshop on Emerging Trends in FLOSS Research and Development (FLOSS'07: ICSE Workshops 2007)*. IEEE, 7–7.
- [35] Audris Mockus, Diomidis Spinellis, Zoe Kotti, and Gabriel John Dusing. 2020. A complete set of related git repositories identified via community detection approaches based on shared commits. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 513–517.
- [36] Ravi Sen, Chandrasekar Subramaniam, and Matthew L Nelson. 2008. Determinants of the choice of open source software license. *Journal of Management Information Systems* 25, 3 (2008), 207–240.
- [37] Sonali K. Shah. 2006. Motivation, Governance, and the Viability of Hybrid Forms in Open Source Software Development. *Management Science* 52, 7 (July 2006), 1000–1014.
- [38] Zakariyah Shoroye, Waheeb Yaqub, Azhar Ahmed Mohammed, Zeyar Aung, and Davor Svetinovic. 2015. Exploring social contagion in open-source communities by mining software repositories. In *International Conference on Neural Information Processing*. Springer, 120–127.
- [39] Katherine J Stewart, Anthony P Ammeter, and Likoebe M Maruping. 2006. Impacts of license choice and organizational sponsorship on user interest and development activity in open source software projects. *Information Systems Research* 17, 2 (2006), 126–144.
- [40] Jason Tsay, Laura Dabbish, and James Herbsleb. 2014. Influence of social and technical factors for evaluating contribution in GitHub. In *Proceedings of the 36th international conference on Software engineering*. 356–366.
- [41] Georg von Krogh, Stefan Haefliger, Sebastian Spaeth, and Martin W. Wallin. 2012. Carrots and Rainbows: Motivation and Social Practice in Open Source Software Development. *MIS Quarterly* 36, 2 (2012), 649–676. <http://www.jstor.org/stable/41703471>
- [42] Patrick Wagstrom, James D Herbsleb, Robert E Kraut, and Audris Mockus. 2010. The impact of commercial organizations on volunteer participation in an online community. In *Academy of Management Annual Meeting*. 1–40.
- [43] Thomas Wolter, Ann Barcomb, Dirk Riehle, and Nikolay Harutyunyan. 2023. Open source license inconsistencies on github. *ACM Transactions on Software Engineering and Methodology* 32, 5 (2023), 1–23.
- [44] Jiaqi Wu, Lingfeng Bao, Xiaohu Yang, Xin Xia, and Xing Hu. 2024. A Large-Scale Empirical Study of Open Source License Usage: Practices and Challenges. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*. IEEE, 595–606.
- [45] Sihan Xu, Ya Gao, Lingling Fan, Zheli Liu, Yang Liu, and Hua Ji. 2023. Lidetector: License incompatibility detection for open source software. *ACM Transactions on Software Engineering and Methodology* 32, 1 (2023), 1–28.
- [46] Weiwei Xu, Kai Gao, Hao He, and Minghui Zhou. 2025. LiCoEval: Evaluating LLMs on License Compliance in Code Generation. In *Proceedings of the 47th International Conference on Software Engineering*.
- [47] Zhiyuan Yu, Yuhao Wu, Ning Zhang, Chenguang Wang, Yevgeniy Vorobeychik, and Chaowei Xiao. 2023. Codeiprompt: Intellectual property infringement assessment of code language models. In *International Conference on Machine*

1277	<i>Learning</i> . PMLR, 40373–40389.	1335
1278	[48] Minghui Zhou, Audris Mockus, Xiujuan Ma, Lu Zhang, and Hong Mei. 2016.	1336
1279	Inflow and retention in oss communities with commercial involvement: A case	1337
1280	study of three hybrid projects. <i>ACM Transactions on Software Engineering and</i>	1338
1281	<i>Methodology (TOSEM)</i> 25, 2 (2016), 13. https://dl.acm.org/authorize?N14201	1339
1282		1340
1283		1341
1284		1342
1285		1343
1286		1344
1287		1345
1288		1346
1289		1347
1290		1348
1291		1349
1292		1350
1293		1351
1294		1352
1295		1353
1296		1354
1297		1355
1298		1356
1299		1357
1300		1358
1301		1359
1302		1360
1303		1361
1304		1362
1305		1363
1306		1364
1307		1365
1308		1366
1309		1367
1310		1368
1311		1369
1312		1370
1313		1371
1314		1372
1315		1373
1316		1374
1317		1375
1318		1376
1319		1377
1320		1378
1321		1379
1322		1380
1323		1381
1324		1382
1325		1383
1326		1384
1327		1385
1328		1386
1329		1387
1330		1388
1331		1389
1332		1390
1333		1391
1334		1392